

Istacherni

Hybrid RAG Architecture Roadmap

Complete Pipeline:

Data Import -> BM25 -> BGE-M3 -> RRF Fusion

-> Cross-Encoder Reranking -> Gemma2:9b -> Dual Evaluation

Algerian Legal QA System

Fine-tuned BGE-M3 + BAAI/bge-reranker-v2-m3 + Gemma2:9b via Ollama

P@1 = 82.68% | MRR = 0.8731 | Hit@5 = 94.49% | NDCG@5 = 0.8910

Stages: 9

P@1: 82.68%

MRR: 0.8731

alpha*: 0.70

Generated from source code analysis - May 2026

Istacherni -- Complete Hybrid RAG Architecture Roadmap

System goal: Given an Arabic legal question, retrieve the single most relevant Algerian law article and produce a grounded, citation-rich answer in Modern Standard Arabic using Gemma2:9b.

Stage 0 -- Raw Data Import & Normalisation

Module: bm25_rag/bm25_loader.py -> load_all_articles()

What happens

Every .json file under dataset/raw/* is walked recursively. Each file may follow one of three schemas:

- * Schema A - {"text": {"original": "..."}} (nested dict)
- * Schema B - {"text": "..."} (flat string)
- * Schema C - {"text_original": "..."} (pre-flattened)

The loader unifies all three into a canonical article dict:

```
{
  id, law_domain, law_name, article_number,
  title, text_original, text_explanation,
  summary, keywords, penalties_summary,
  legal_conditions_summary
}
```

Deduplication rule

Articles sharing the same id but different (text_original, text_explanation) pairs are kept as separate versions with a numeric suffix (ART_42_1, ART_42_2, ...).

Reference

Robertson, S. & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. *Foundations and Trends in IR*, 3(4), 333-389.

Stage 1 -- BM25 Sparse Index

Module: bm25_rag/bm25_indexer.py -> build_index()

Library: ra

1.1 Arabic Tokenisation

```
def tokenize_arabic(text):
    text = re.sub(r'[\u064B-\u065F\u0640]', '', text) # strip diacritics + tatweel
    text = re.sub(r'[[ar]]', '[ar]', text)           # normalise alef variants
    text = re.sub(r'[ar]', '[ar]', text)             # teh marbuta -> ha
    text = re.sub(r'[ar]', '[ar]', text)             # alef maqsura -> ya
    tokens = re.split(r'^\w\u0600-\u06FF+', text.lower())
    return [t for t in tokens if len(t) >= 2]
```

This normalisation prevents vocabulary explosion from orthographic variants common in Arabic legal text.

1.2 Document Enrichment

Each article is expanded into a rich string before tokenisation. Keywords are repeated twice to double their term-frequency contribution without modifying the BM25 formula:

```
doc_text = header + title + text_original + summary
          + legal_conditions_summary + penalties_summary
          + keywords + keywords          <- x2 boost
```

1.3 BM25 Okapi Formula

For a query $Q = \{q_1, \dots, q_n\}$ and document D :

$$\text{BM25}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1} \cdot \left(\frac{1 - b + b \cdot \frac{|D|}{\text{avgdl}}}{\frac{|D|}{\text{avgdl}}} \right)$$

where:

- * $f(q_i, D)$ = term frequency of q_i in D
- * $|D|$ = document length (tokens)
- * avgdl = average document length across corpus
- * $k_1 = 1.5$, $b = 0.75$ (BM25Okapi defaults)
- * $\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$

1.4 Index Persistence

The fitted BM25Okapi object and the ordered corpus list are pickled to:

- * `bm25_rag/index/bm25_index.pkl`
- * `bm25_rag/index/bm25_corpus.pkl`

Subsequent calls load from disk (lazy singleton via `_get_index()`).

1.5 Retrieval call

```
bm25_results = bm25_retrieve(question, top_k=100)
```

Returns the top-100 articles sorted by BM25 score (descending), with score ≤ 0 filtered out.

Reference: Trotman, A. et al. (2014). Improvements to BM25 and Language Models Examined. ADCS.

Stage 2 -- Dense Semantic Index (BGE-M3)

Module: `dense_rag/bge_embedder.py` + `dense_rag/bge_indexer.py`

Vector Sto

2.1 Fine-Tuned Embedding Model

The system uses a domain-adapted version of BAAI/bge-m3, fine-tuned on Algerian legal triplets and saved at:

```
bge_m3_algerian_law_v3/
```

The model is loaded via sentence-transformers on CPU (GPU is reserved exclusively for the LLM):

```
_model = SentenceTransformer(str(MODEL_PATH), device="cpu")
```

2.2 Searchable String Construction

Each article is represented as a structured concatenation to maximise semantic coverage:

```
[law_name - [ar] art_num] | title | keywords | text_original | text_explanation | summary
```

2.3 Query Embedding with Instruction Prefix

BGE-M3 supports asymmetric retrieval via a query instruction prefix:

```
prefixed = "Represent this query for retrieving relevant documents: " + query
embedding = model.encode(prefixed, normalize_embeddings=True)
```

Embeddings are L2-normalised so that inner product equals cosine similarity.

2.4 Cosine Similarity in ChromaDB

ChromaDB is configured with `hnsw:space = cosine`. It returns cosine distance $d \in [0, 2]$, which the retriever converts to a similarity score:

$$\text{score} = 1 - d_{\cos}(q, d)$$

where $d_{\cos}(q, d) = 1 - \frac{q \cdot d}{\|q\| \|d\|}$.

Since embeddings are L2-normalised, $\|q\| = \|d\| = 1$, so:

$$\text{score} = q \cdot d$$

2.5 Retrieval call

```
dense_results = dense_retrieve(question, top_k=100)
```

Returns the top-100 articles sorted by cosine similarity.

References:

- Chen, J. et al. (2024). BGE M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv:2309.07597*.
- Johnson, J. et al. (2021). Billion-Scale Similarity Search with GPUs. *IEEE TBIG*.

Stage 3 -- Reciprocal Rank Fusion (RRF)

Module: `hybrid_rag/hybrid_retriever.py` -> `reciprocal_rank_fusion()`

Formula

RRF combines two ranked lists by summing rank-based scores, making it robust to score-scale differences between BM25 and cosine similarity:

$$\text{RRF}(d) = \sum_{r \in \{\text{BM25}, \text{Dense}\}} w_r \cdot \frac{1}{k + \text{rank}(d, r)}$$

where:

- * $k = 60$ (smoothing constant, standard default)
- * $\text{rank}(d, r)$ = 1-indexed rank of document d in ranker r
- * $w_{\text{BM25}} = 0.3$, $w_{\text{Dense}} = 0.7$ (tuned to favour semantic retrieval)

Why $k = 60$?

Cormack et al. (2009) showed $k = 60$ provides the best empirical performance across diverse IR tasks. It prevents rank-1 documents from dominating through a score of $\frac{1}{61} \approx 0.016$ rather than 1.0 .

Implementation

```
for rank, article in enumerate(bm25_results, 1):
    combined_scores[aid] += bm25_weight * (1.0 / (k + rank))

for rank, article in enumerate(dense_results, 1):
    combined_scores[aid] += dense_weight * (1.0 / (k + rank))
```

The top-30 fused candidates proceed to the reranker.

Reference: Cormack, G.V., Clarke, C.L.A., & Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009.

Stage 4 -- Fused Reranking

Module: hybrid_rag/reranker.py -> rerank_candidates()

The reranker applies two scoring functions and fuses them, controlled by FUSION_ALPHA = 0.7.

4.1 Scorer A -- BGE-M3 Unified (Multi-Granularity)

The BGE-M3 model computes a unified score from three retrieval modes in a single forward pass:

$$S_{\text{BGE-M3}}(q, d) = 0.4 \cdot S_{\text{ColBERT}}(q, d) + 0.2 \cdot S_{\text{Sparse}}(q, d) + 0.4 \cdot S_{\text{Dense}}(q, d)$$

ColBERT (late-interaction):

Sparse (term-matching):

Dense (single-vector):

```
scores_dict = bge_model.compute_score_single_device(
    pairs,
    weights_for_different_modes=[0.4, 0.2, 0.4],
    device="cpu"
)
raw_scores_A = scores_dict["colbert+sparse+dense"]
```

4.2 Scorer B -- Fine-Tuned Cross-Encoder

A BAAI/bge-reranker-v2-m3 cross-encoder fine-tuned on domain data, saved at reranker_finetuned/.

The cross-encoder concatenates [CLS] query [SEP] document [SEP] and produces a single relevance logit:

$$s_{\text{CE}}(q, d) = \sigma(W^{\text{top}} \cdot \text{BERT}([q; d])_{\text{CLS}})$$

with σ = sigmoid, output $\in (0, 1)$.

Query prompt applied at inference (consistent with training):

```
"Represent this sentence for searching relevant passages: {query}"
```

4.3 Min-Max Normalisation

Both score vectors are normalised to $[0, 1]$ independently before fusion:

$$\hat{s}_i = \frac{s_i - \min_j s_j}{\max_j s_j - \min_j s_j}$$

If all scores are equal, every document gets \$0.5\$.

4.4 Score Fusion

$$\text{score}_{\text{final}}(d) = \alpha \cdot \text{score}_{\text{BGE-M3}}(d) + (1 - \alpha) \cdot \text{score}_{\text{CE}}(d)$$

with $\alpha = 0.7$ (optimal from alpha-sweep, see Stage 8).

The top-5 documents by fused score are passed to the LLM.

References:

- Nogueira, R. & Cho, K. (2019). Passage Re-ranking with BERT. arXiv:1901.04085.

- Chen, J. et al. (2024). BGE M3-Embedding. arXiv:2309.07597.

Stage 5 -- Cross-Encoder Fine-Tuning

Module: finetune_reranker.py

Base mod

5.1 Training Data

Pairs were prepared by prepare_reranker_data.py from hard_negatives.jsonl (70 priority triplets plus mined negatives):

```
{"query": "...", "document": "...", "label": 1.0} // positive
{"query": "...", "document": "...", "label": 0.0} // negative
```

5.2 Loss Function -- Weighted BCEWithLogitsLoss

To address class imbalance (many more negatives than positives):

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \left[w_+ \log \sigma(s_i) + (1 - y_i) \log(1 - \sigma(s_i)) \right]$$

where:

$$w_+ = \frac{\text{negatives}}{\text{positives}}$$

5.3 Hyperparameters

Parameter	Value	Rationale
Learning rate	2×10^{-5}	Per GEMINI.md protocol
Epochs	3	Early stopping patience=5
Batch size	4	6 GB VRAM constraint
Grad accumulation	8	Effective batch = 32
Optimiser	Adafactor	Replaces AdamW (~4.5 GB) -> ~0.5 GB
Warmup ratio	0.1	Stable LR ramp
Max seq length	512	
Weight decay	0.01	

5.4 Triplet Evaluation (Legal Discrimination)

After every epoch, a CrossEncoderTripletEvaluator measures:

$$\text{TripletAccuracy} = \frac{1}{|T|} \sum_{(a,p,n) \in T} \mathbb{1}[s(a,p) > s(a,n)]$$

This directly measures the model's ability to rank gold articles above hard negatives.

References:

- Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers. NAACL.

- Shazeer, N. & Stern, M. (2018). Adafactor: Adaptive Learning Rates with Sublinear Memory Cost. ICML.

Stage 6 -- LLM Generation (Gemma2:9b)

Module: run_fused_hybrid_generation.py -> run_query_with_timing()

Interface:

6.1 Context Assembly

The top-5 reranked documents are formatted as numbered articles:

```
[ar] 1:
[law_name - [ar] art_num] title. text_original text_explanation summary keywords

[ar] 2: ...
```

6.2 Prompt Structure

A system prompt enforces Chain-of-Thought reasoning and citation obligation:

```
[ar] [ar] [ar] [ar] [ar] [ar] [ar] [ar].
[ar] [ar]:
1. [ar] [ar] [ar] [ar] [ar]
2. [ar] [ar] [ar] (Chain of Thought)
3. [ar] [ar]: [[ar] {[ar]}][ar] {[ar] [ar]}]
4. [ar] [ar] [ar] [ar] [ar] [ar] [ar]
[ar]: ([ar]) [ar] [ar] ([ar]) [ar] [ar] ([ar]) [ar] ([ar]) [ar]
```

The user message template:

```
[ar] [ar] [ar] [ar] [ar] [ar]:
{context_text}
[ar]: {question}
[ar]:
```

6.3 Generation Parameters

Parameter	Value	Rationale
`num_gpu`	28 layers	Partial GPU offload to fit 6 GB VRAM
`num_thread`	6	CPU handles remaining 14 layers
`num_ctx`	2048	Safe context window for VRAM
`temperature`	0.1	Near-deterministic for legal accuracy
`top_p`	0.9	Nucleus sampling
`stream`	False	Full response before writing
Timeout	180 s	

Reference: Team, G. et al. (2024). Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118.

Stage 7 -- Retrieval Evaluation

7.1 Article Matching Logic

For each retrieved chunk, article number and law name are compared to ground-truth. A match requires:

- * Article numbers identical (string equality)
- * Law names overlap (substring or contain [ar])

7.2 Metrics

Precision@1 (Top-1 Accuracy)

$$P@1 = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}[\text{rank}_1 = \text{gold} \mid \text{right}]$$

Mean Reciprocal Rank (MRR)

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_{\text{first hit}}(q)}$$

If no hit in top-30: $\frac{1}{\text{rank}} = 0$.

MRR is the primary metric per the GEMINI.md protocol -- the correct article must rank #1.

Hit Rate @ K

$$\text{HR}@K = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}[\exists r \leq K : \text{match}(r, q)]$$

Recall @ 30

$$\text{Recall}@30 = \frac{|\text{matched GT articles}|}{|\text{total GT articles}|}$$

7.3 Results (Hybrid BGE-M3 + BM25, post-fusion-reranking)

Metric	Value
Precision@1	82.68%**
MRR	0.8731**
Hit Rate @5	94.49%**
NDCG@5	0.8910**

Reference: Voorhees, E.M. (1999). The TREC-8 Question Answering Track Report. TREC.

Stage 8 -- Alpha Sweep (Reranker Fusion Calibration)

Module: fuse_reranker.py

The optimal fusion weight α is found by exhaustive sweep $\alpha \in \{0.0, 0.1, \dots, 1.0\}$:

$$\text{score}(d, \alpha) = \alpha \cdot \hat{S}_A(d) + (1 - \alpha) \cdot \hat{S}_B(d)$$

Sweep Results

alpha	P@1	MRR	Hit@5	NDCG@5
0.0 (CE only)	77.17%	0.8371	92.91%	0.8604
0.3	79.53%	0.8517	92.91%	0.8714
0.5	80.31%	0.8600	94.49%	0.8812
0.7**	82.68%**	0.8731**	94.49%**	0.8910**
0.8	81.89%	0.8676	93.70%	0.8850

1.0 (BGE only)	79.53%	0.8425	89.76%	0.8568
----------------	--------	--------	--------	--------

Optimal: $\alpha = 0.7$ -> +3.15% P@1 over BGE-M3 alone.

Stage 9 -- LLM Generation Evaluation (RAGAS-Style)

Module: evaluate_ragas.py

9.1 Metrics

Answer Similarity -- semantic cosine similarity via multilingual SBERT:

$$\text{Sim}(a, \hat{a}) = \frac{E_a \cdot E_{\hat{a}}}{\|E_a\| \|E_{\hat{a}}\|}$$

Answer Correctness -- token-level F1 (after Arabic normalisation):

$$F1 = \frac{2 \cdot P \cdot R}{P + R}, \quad P = \frac{|a \cap \hat{a}|}{|\hat{a}|}, \quad R = \frac{|a \cap \hat{a}|}{|a|}$$

Faithfulness -- fraction of answer tokens covered by ground-truth:

$$\text{Faith}(a, \hat{a}) = \frac{|\bigcup_{t \in \text{tokens}(\hat{a})} \text{tokens}(a)|}{|\text{tokens}(\hat{a})|}$$

Article Recall -- fraction of ground-truth article numbers cited:

$$\text{ArtRecall} = \frac{|\text{cited}_{\text{pred}} \cap \text{cited}_{\text{GT}}|}{|\text{cited}_{\text{GT}}|}$$

9.2 Weighted Accuracy

$$\text{Accuracy} = 0.35 \cdot \text{Sim} + 0.30 \cdot \text{F1} + 0.20 \cdot \text{Faith} + 0.15 \cdot \text{ArtRecall}$$

9.3 Arabic Normalisation (applied to all text before scoring)

```
text = re.sub(r'[\u0617-\u061A\u064B-\u065F]', '', text) # remove diacritics
text = re.sub(r'[[ar]]', '[ar]', text) # unify alef
text = text.replace('[ar]', '[ar]') # teh marbuta
text = text.replace('[ar]', '[ar]') # alef maqsura
text = re.sub(r'^\w\s', '', text) # strip punctuation
```

9.4 Embedding Model Comparison Results

Model	Similarity	Correctness	Faithfulness	Art. Recall	**Accuracy**
BGE-M3 (ours)	highest	highest	highest	highest	**best**
MiniLM	--	--	--	--	2nd
Qwen	--	--	--	--	3rd
CamELBERT	--	--	--	--	4th

References:

- Es, S. et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217.
- Rajpurkar, P. et al. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. EMNLP.

Full Pipeline Data-Flow

```
Raw JSON files (dataset/raw/**)
    ?
    ?
[Stage 0] bm25_loader.load_all_articles()
    -> Normalised article dicts (deduplicated)
    ?
    ??????????
    ?      ?
[Stage 1] [Stage 2]
BM250kapi  BGE-M3 fine-tuned
Arabic tok  L2-norm embeddings
pickle idx  ChromaDB HNSW
    ?      ?
    ? top-100 ? top-100
    ??????????
    ?
[Stage 3] Reciprocal Rank Fusion
    RRF(d) = 0.3/(60+rank_BM25) + 0.7/(60+rank_Dense)
    -> top-30 fused candidates
    ?
    ?
[Stage 4] Fused Reranker
    Scorer A: BGE-M3 Unified (ColBERT+Sparse+Dense, weights 0.4/0.2/0.4)
    Scorer B: Fine-Tuned Cross-Encoder (BCEWithLogitsLoss)
    Normalise both -> [0,1]
    Final = 0.7.A + 0.3.B?
    -> top-5 articles
    ?
    ?
[Stage 6] Gemma2:9b via Ollama
    System prompt (CoT + citation rules)
    User prompt (top-5 articles + question)
    temp=0.1, ctx=2048, 28 GPU layers
    -> Arabic legal answer
    ?
    ?
[Stage 7] Retrieval Eval          [Stage 9] LLM Eval
    P@1, MRR, HR@30, NDCG@5      Sim, F1, Faith, ArtRecall
    CSV + terminal report         Radar chart PNG + CSV
```

Hardware Configuration

Component	Spec	Notes
GPU	NVIDIA (6 GB VRAM)	28 layers of Gemma2:9b
CPU	6-thread	14 remaining LLM layers + all retrieval
BGE-M3 Embedder	CPU	Avoid VRAM conflict
BGE-M3 Reranker	CPU	`use_gpu=False`
Cross-Encoder	CPU	fp32 inference
Gemma2:9b	GPU (partial)	Served via Ollama

Reference: Frantar, E. et al. (2022). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. [arXiv:2210.17323](#).

Key References

- * Robertson, S. & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in IR*, 3(4).
- * Cormack, G.V., Clarke, C.L.A., & Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *SIGIR 2009*.
- * Chen, J. et al. (2024). BGE M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings. *arXiv:2309.07597*.
- * Nogueira, R. & Cho, K. (2019). Passage Re-ranking with BERT. *arXiv:1901.04085*.
- * Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT 2019*.
- * Shazeer, N. & Stern, M. (2018). Adafactor: Adaptive Learning Rates with Sublinear Memory Cost. *ICML 2018*.
- * Team, G. et al. (2024). Gemma 2: Improving Open Language Models at a Practical Size. *arXiv:2408.00118*.
- * Es, S. et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv:2309.15217*.
- * Voorhees, E.M. (1999). The TREC-8 Question Answering Track Report. *TREC*.
- * Johnson, J. et al. (2021). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*.